

SOF 103 Week 3: Operators and Control Structures

Generated by Review Agent on 2026-07-02 06:34 UTC

Source: SOF 103-Chapter 03- Operators and Control Structures2024_09.ppt

Classification confidence: 100%

1. Core Knowledge Outline

中文

• 1.1 运算符：程序的“动词”

- **算术运算符**：执行数学计算（`+`，`-`，`*`，`/`，`%`）。
 - `%`（取模/取余）是理解整数运算的关键。
- **赋值运算符**：将值存储到变量中（`=`）。
 - `x = 5` 表示“将5赋值给x”，而非“x等于5”。
 - **复合赋值运算符**：简化代码（`+=`，`-=`，`*=` 等）。`x += 1` 等价于 `x = x + 1`。
- **比较运算符**：比较两个值，返回布尔结果（`True` 或 `False`）（`==`，`!=`，`>`，`<`，`>=`，`<=`）。
 - `==`（等于）与 `=`（赋值）是初学者最常见的混淆点。
- **逻辑运算符**：组合多个布尔条件（`and`，`or`，`not`）。
 - `and`：所有条件为真才为真。
 - `or`：至少一个条件为真即为真。
 - `not`：反转布尔值。
- **运算符优先级**：决定表达式中运算的顺序（例如，乘除优先于加减）。
 - 使用括号 `()` 可以明确并覆盖默认优先级。

- **1.2 控制结构：程序的“决策者”与“循环者”**

- **顺序结构**：代码默认从上到下、逐行执行。
- **选择（分支）结构**：根据条件决定执行哪段代码。
 - `if` 语句：最基本的条件判断。
 - `if-else` 语句：二选一。
 - `if-elif-else` 语句：多选一。
- **循环结构**：重复执行一段代码块。
 - `for` 循环：用于**已知**循环次数或遍历一个序列（如列表、字符串）。
 - `while` 循环：用于**未知**循环次数，只要条件为真就继续执行。
- **循环控制**：改变循环的正常执行流程。
 - `break`：立即**终止**整个循环。
 - `continue`：**跳过**当前循环的剩余部分，进入下一次迭代。

English

- **1.1 Operators: The "Verbs" of a Program**

- **Arithmetic Operators**: Perform mathematical calculations (`+` , `-` , `*` , `/` , `%`).
 - `%` (Modulo/Remainder) is key to understanding integer division.
- **Assignment Operators**: Store values into variables (`=`).
 - `x = 5` means "assign 5 to x", not "x equals 5".
 - **Compound Assignment Operators**: Simplify code (`+=` , `-=` , `*=` etc.).
`x += 1` is equivalent to `x = x + 1`.
- **Comparison Operators**: Compare two values, returning a Boolean result (`True` or `False`) (`==` , `!=` , `>` , `<` , `>=` , `<=`).
 - `==` (equal to) vs `=` (assignment) is the most common beginner confusion.
- **Logical Operators**: Combine multiple Boolean conditions (`and` , `or` , `not`).
 - `and` : True only if all conditions are true.
 - `or` : True if at least one condition is true.

- `not` : Reverses the Boolean value.
- **Operator Precedence**: Determines the order of operations in an expression (e.g., multiplication/division before addition/subtraction).
 - Use parentheses `()` to explicitly override default precedence.
- **1.2 Control Structures: The "Decision-Makers" and "Loopers" of a Program**
 - **Sequential Structure**: Code executes line by line, from top to bottom by default.
 - **Selection (Branching) Structure**: Decides which block of code to execute based on a condition.
 - `if` statement: The most basic conditional check.
 - `if-else` statement: Choose one of two paths.
 - `if-elif-else` statement: Choose one of many paths.
 - **Looping Structure**: Repeats a block of code.
 - `for` loop: Used when the number of iterations is **known** or to iterate over a sequence (e.g., list, string).
 - `while` loop: Used when the number of iterations is **unknown**; it continues as long as a condition is true.
 - **Loop Control**: Alters the normal flow of a loop.
 - `break` : **Terminates** the entire loop immediately.
 - `continue` : **Skips** the rest of the current iteration and proceeds to the next one.

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
运算符 (Operator)	一种特殊的符号或关键字，用于对一个或多个值（操作数）执行操作，并产生一个结果。	中文：就像数学课上的加号（+）和减号（-），它们是程序的“动作指令”。 English: Like the plus (+) and minus (-) signs in math class; they are the "action commands" for a program.
操作数 (Operand)	运算符所操作的值或变量。	中文：在 <code>5 + 3</code> 中，<code>5</code> 和 <code>3</code> 就是操作数，它们是运算符“工作”的对象。 English: In <code>5 + 3</code>, <code>5</code> and <code>3</code> are the operands; they are the objects the operator "works on".
布尔表达式 (Boolean Expression)	一个计算结果为布尔值（ <code>True</code> 或 <code>False</code> ）的表达式，通常由比较运算符或逻辑运算符构成。	中文：就像是一个“是非题”，答案只能是“对”或“错”。例如 <code>age ≥ 18</code> 就是一个布尔表达式。 English: Like a "true or false" question; the answer can only be "correct" or "wrong". For example, <code>age ≥ 18</code> is a Boolean expression.
条件语句 (Conditional Statement)	一种控制结构，它根据布尔表达式的值（真或假）来决定是否执行特定的代码块。	中文：就像生活中的“如果...那么...”。例如，“如果下雨，那么我就带伞”。 English: Like an "if...then..." in real life. For example, "If it rains, then I will take an umbrella."
循环 (Loop)	一种控制结构，允许一段代码被重复执行多次，直到满足某个特定条件为止。	中文：就像跑步时绕操场跑圈，直到跑完规定的圈数（<code>for</code> 循环）或听到哨声（<code>while</code> 循环）才停下来。 English: Like running laps around a track until you've completed a set number (a <code>for</code> loop) or until you hear a whistle (a <code>while</code> loop).
迭代 (Iteration)	循环执行一次代码块的过程。	中文：循环体每执行一次，就称为一次“迭代”。就像跑完一圈。 English: One single execution of the loop's code block. Like completing one lap.

Term	Definition	Memory Anchor / Analogy
break 语句	用于立即终止当前所在的循环，程序将继续执行循环之后的代码。	中文： 就像在跑步时，突然接到命令“全体停止！”，无论跑了多少圈，立刻停下。 English: Like getting a command "Everyone stop!" while running; you stop immediately, no matter how many laps you've done.
continue 语句	用于跳过当前迭代中剩余的代码，并立即开始下一次循环迭代。	中文： 就像在接力赛中，你这一棒跑得不好，教练让你直接跳过，换下一个队友跑下一棒。 English: Like in a relay race where you're having a bad leg; the coach tells you to skip your turn and let the next teammate run the next leg.
运算符优先级 (Operator Precedence)	定义在包含多个不同运算符的表达式中，哪个运算应该首先被执行的规则集。	中文： 就像数学中的“先乘除，后加减”。 <code>2 + 3 * 4</code> 结果是 <code>14</code> ，而不是 <code>20</code> 。 English: Like "multiplication and division before addition and subtraction" in math. The result of <code>2 + 3 * 4</code> is <code>14</code> , not <code>20</code> .

3. Critical Takeaways

中文

- 区分 `=` 和 `==`：**这是编程新手最常见的错误。`=` 是**赋值**，用于将右边的值存入左边的变量。`==` 是**比较**，用于检查左右两边的值是否相等，并返回 `True` 或 `False`。考试中一定会考到这一点。
- 理解 `if-elif-else` 的短路行为：**在 `if-elif-else` 链中，一旦某个条件为 `True`，其对应的代码块将被执行，然后**整个 `if-elif-else` 结构就会结束**。后续的 `elif` 和 `else` 将**不再被检查**。这是高效编程的关键。
- 警惕无限循环：**使用 `while` 循环时，必须确保循环条件最终会变为 `False`。否则，程序将永远运行下去（无限循环），导致程序卡死。常见的错误是忘记在循环体内更新用于判断条件的变量。

4. **for 循环 vs while 循环的选择**: 当你**知道**要循环多少次, 或者需要遍历一个集合 (如列表) 时, 使用 `for` 循环。当你**不知道**要循环多少次, 但知道在什么条件下应该停止时, 使用 `while` 循环。选错循环类型是常见的逻辑错误。
5. **善用括号 () 控制优先级**: 当表达式中混合了多种运算符, 或者你不确定优先级时, **总是使用括号 () 来明确运算顺序**。这不仅能避免错误, 还能让你的代码意图更清晰, 便于他人阅读。

English

1. **Distinguish `=` and `==`**: This is the most common mistake for new programmers. `=` is **assignment**, used to store the value on the right into the variable on the left. `==` is **comparison**, used to check if the values on both sides are equal, returning `True` or `False`. This will definitely be on the exam.
2. **Understand `if-elif-else` Short-Circuiting**: In an `if-elif-else` chain, once a condition is `True`, its corresponding block executes, and then the **entire `if-elif-else` structure ends**. Subsequent `elif` and `else` conditions are **no longer checked**. This is key for efficient programming.
3. **Beware of Infinite Loops**: When using a `while` loop, you must ensure the loop condition will eventually become `False`. Otherwise, the program will run forever (an infinite loop), causing it to freeze. A common mistake is forgetting to update the variable used in the condition inside the loop body.
4. **Choosing `for` vs `while`**: Use a `for` loop when you **know** how many times you need to iterate, or you need to traverse a collection (like a list). Use a `while` loop when you **don't know** how many times you'll need to iterate, but you know the condition under which to stop. Choosing the wrong loop type is a common logical error.
5. **Use Parentheses `()` to Control Precedence**: When an expression mixes multiple operators, or you are unsure about precedence, **always use parentheses `()` to make the order of operations explicit**. This not only prevents errors but also makes your code's intention clearer and easier for others to read.

4. Detailed Notes (AI-Expanded)

4.1 运算符：构建程序逻辑的基石 / Operators: The Building Blocks of Program Logic

中文

运算符是编程语言中最基本的元素之一，它们允许我们对数据进行操作。可以把它想象成语言中的“动词”，而数据（变量、常量）是“名词”。没有动词，句子就没有动作；没有运算符，程序就无法进行计算、比较和决策。

- **算术运算符**：这是你最熟悉的，用于执行基本的数学运算。
 - `+` (加), `-` (减), `*` (乘), `/` (除)。
 - `%` (取模/取余)：这个运算符非常有用，它返回除法运算的余数。例如，`7 % 2` 的结果是 `1`，因为7除以2得3余1。它常用于判断一个数是奇数还是偶数（`num % 2 == 0` 为偶数），或者将一个数字限制在特定范围内。
 - `**` (幂运算)：例如 `2 ** 3` 等于 `8`。
 - `//` (整数除法/地板除)：例如 `7 // 2` 的结果是 `3`，它只保留商的整数部分，丢弃小数部分。
- **赋值运算符**：用于将值“放入”变量中。
 - 基本赋值 `=`： `my_variable = 10`。
 - 复合赋值运算符：它们是基本算术运算符和赋值运算符的组合，用于简化代码。
 - `x += 5` 等价于 `x = x + 5`。
 - `y *= 2` 等价于 `y = y * 2`。
 - 这种写法更简洁，也更能表达“对变量自身进行更新”的意图。
- **比较运算符**：用于比较两个值，结果总是一个布尔值（`True` 或 `False`）。
 - `==` (等于)：检查两个值是否相等。**再次强调，不要与 `=` 混淆。**
 - `!=` (不等于)：检查两个值是否不相等。
 - `>` (大于), `<` (小于), `>=` (大于等于), `<=` (小于等于)。
 - 比较运算符是构成条件判断（`if` 语句）和循环（`while` 语句）的基础。
- **逻辑运算符**：用于组合多个布尔表达式，形成更复杂的条件。

- `and` (与): 当且仅当所有条件都为 `True` 时, 结果才为 `True`。例如, `is_weekend and is_sunny` 只有在既是周末又是晴天时才为真。
- `or` (或): 只要有一个条件为 `True`, 结果就为 `True`。例如, `is_teacher or is_admin` 只要用户是老师或者是管理员, 就为真。
- `not` (非): 用于反转一个布尔值。 `not True` 等于 `False`, `not False` 等于 `True`。例如, `not is_raining` 表示“没有下雨”。
- **运算符优先级**: 当一个表达式包含多种运算符时, Python 会按照一套预定义的规则 (优先级) 来决定先执行哪个运算。
 - **记忆口诀: PEMDAS** (括号 Parentheses, 指数 Exponents, 乘除 Multiplication/Division, 加减 Addition/Subtraction)。逻辑运算符的优先级通常低于比较运算符。
 - **最佳实践: 不要依赖记忆来记住复杂的优先级规则**。当你不确定时, 或者想让代码更清晰时, **请使用括号 `()` 来明确指定运算顺序**。例如, `(a + b) * c` 比 `a + b * c` 清晰得多, 即使后者在数学上也是先算乘法。

English

Operators are one of the most fundamental elements in a programming language; they allow us to manipulate data. Think of them as the "verbs" in a language, while data (variables, constants) are the "nouns." Without verbs, a sentence has no action; without operators, a program cannot perform calculations, comparisons, or make decisions.

- **Arithmetic Operators**: These are the ones you're most familiar with, used for basic mathematical operations.
 - `+` (Addition), `-` (Subtraction), `*` (Multiplication), `/` (Division).
 - `%` (Modulo/Remainder): This operator is very useful; it returns the remainder of a division. For example, `7 % 2` results in `1` because 7 divided by 2 is 3 with a remainder of 1. It's commonly used to check if a number is odd or even (`num % 2 == 0` means even), or to confine a number to a specific range.
 - `**` (Exponentiation): For example, `2 ** 3` equals `8`.

- `//` (Integer Division/Floor Division): For example, `7 // 2` results in `3`. It keeps only the integer part of the quotient, discarding the fractional part.
- **Assignment Operators:** Used to "put" a value into a variable.
 - Basic assignment `=`: `my_variable =`